Name: Muhammad Abdul Kareem

Roll No: BSEF23M014

Section: BS-SE(Morning)

Semester: 6^{th}

Assignment: 2^{nd}

Subject: Operating System

Submission Date: 05/11/2026

Submitted to:_____

Linux System Call Interface

Definition

A ***system call*** is a method through which a user program communicates with the operating system kernel to request services such as:

- Process creation
- File handling
- Memory management
- Input/Output operations.

System calls act as an interface between:

- **User Space**
- **Kernel Space**

Purpose

System calls are used for the following:

- Access hardware resources
- Create and manage processes.
- Perform file operations.
- Handle communication between processes.

GNU gcc Compiler

Definition

GNU Compiler is a compiler used to compile C, C++, and other programming languages in Linux.

Basic gcc Commands and Options

The basic GCC commands and options for the gcc compiler are as follows:

1. Compiling a C Program ***gcc program.c***
2. Specify Output File Name ***gcc program.c -o output***

3. Command for running the Program ***./output***
4. Compiling with Warnings ***gcc -Wall program.c -o output***
5. Compiling with Debugging ***gcc -g program.c -o output***
6. Compile pthread Program ***gcc thread.c -o thread -pthread***

Process Creation and Termination

1) getpid()

Purpose

Returns the Process ID (PID) of the current process.

Program

```c
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("Process ID: %d\n", getpid());
    return 0;
}
```

Compile

Compiling the program ***gcc getpid.c -o getpid***

Run

Running the program ***./getpid***

Output

```
Process ID: 742


...Program finished with exit code 0
Press ENTER to exit console.
```

2) getppid()

Purpose

Returns the Parent Process ID (PPID).

Program

```c
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("Parent Process ID: %d\n", getppid());
    return 0;
}
```

Compile

Compiling the program ***gcc getppid.c -o getppid***

Run

Running the program ***./getppid***

Output

```
Parent Process ID: 3865


...Program finished with exit code 0
Press ENTER to exit console.
```

3) fork()

Purpose

Creates a new child process.

Concepts

The important concepts for fork() call are as follows:

- Parent process creates child process.
- Both run independently
- Child gets a copy of parent process.

Program

```c
#include <stdio.h>
#include <unistd.h>

int main() {

    pid_t pid = fork();

    if(pid < 0) {
        printf("Fork Failed\n");
    }

    else if(pid == 0) {
        printf("Child Process\n");
        printf("Child PID: %d\n", getpid());
    }

    else {
        printf("Parent Process\n");
        printf("Parent PID: %d\n", getpid());
    }

    return 0;
}
```

Compile

Compiling the program ***gcc fork.c -o fork***

Run

Running the program ***./fork***

Output

```
Child Process
Child PID: 2238


...Program finished with exit code 0
Press ENTER to exit console.
```

Return Values of fork()

The fork() function returns different values with different conditions, some of which are as follows:

Return Value	Meaning
pid < 0	Fork failed
pid == 0	Child process
pid > 0	Parent process

4) wait()

Purpose

Parent process waits until child process finishes execution.

Program

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {

    pid_t pid = fork();

    if(pid == 0) {
        printf("Child Process Running\n");
    }

    else {
        wait(NULL);
        printf("Parent Process Running After Child Completion\n");
    }

    return 0;
}
```

Compile

Compiling the program ***gcc wait.c -o wait***

Run

Running the program ***./wait***

Output

```
Child Process Running
Parent Process Running After Child Completion


...Program finished with exit code 0
Press ENTER to exit console.
```

5) exit()

Purpose

Terminates a process.

Program

```c
#include <stdio.h>
#include <stdlib.h>

int main() {

    printf("Program Terminated\n");

    exit(0);

    printf("This line will not execute");

    return 0;
}
```

Compile

Compiling the program ***gcc exit.c -o exit***

Run

Running the program ***./exit***

Output

```
Program Terminated


...Program finished with exit code 0
Press ENTER to exit console.
```

6) execl()

Purpose

Replaces current process with another program.

Program

```c
#include <stdio.h>
#include <unistd.h>

int main() {

    printf("Executing ls command\n");

    execl("/bin/ls", "ls", "-l", NULL);

    printf("This will not execute");

    return 0;
}
```

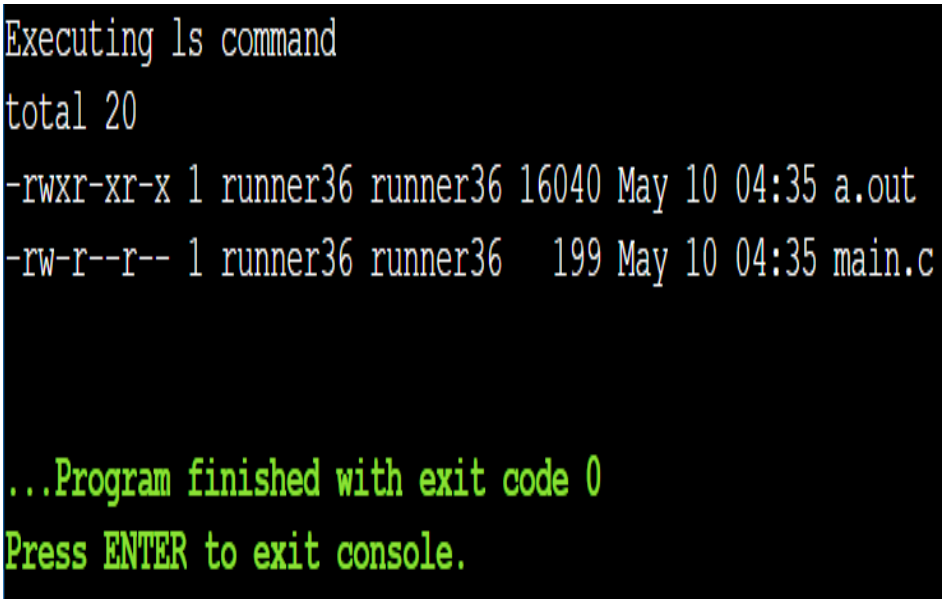
Compile

Compiling the program ***gcc execl.c -o execl***

Run

Running the program ***./execl***

Output



Understanding

Parent Processes

The process that creates another process.

Child Processes

The newly created process using ***fork()***.

Zombie Process

A process that has completed execution but still exists in process table because the parent has not read its exit status.

Cause

When the parent does not use ***wait(NULL)***.

Orphan Process

A child process whose parent process terminates before the child.

Result

The orphan process becomes adopted by ***init/systemd process***.

Inter Process Communication(IPC)

IPC allows processes to communicate and share data.

1) Pipes

Definition

A pipe is used for communication between processes.

Example

The examples for using Pipes on the terminal are as follows:

- ***ls | wc***

Explanation

The explanation for using the above command is as follows:

- ls gives file list.
- wc counts lines/words/characters.

2) FIFO

Definition

It is nothing but a named Pipe.

Create FIFO

- ***mkfifo mypipe***

Write Data

- ***echo "Hello" > mypipe***

Read Data

- ***cat < mypipe***

3) Sockets

Definition

Sockets allow communication between processes over a network.

Types

The types for the sockets are as follows:

- TCP Socket
- UDP Socket

4) IPC Tools in Linux

Tool	Purpose
Pipe	Communication between related processes
FIFO	Communication between unrelated processes
Socket	Network communication
Message Queue	Message-based communication
Shared Memory	Fast shared communication

Linux Pipe Command

Command 1)

- ***cat file.txt | sort***

Command 2)

- ***ps aux | grep firefox***

Mkfifo Command Practice

Create FIFO

- ***mkfifo fifo1***

Check FIFO

- ***ls -l***

Threads and Scheduling

1) pthread_create()

Purpose

Creates a thread.

Program

```c
#include <stdio.h>
#include <pthread.h>

void* display(void* arg) {

    printf("Thread Executing\n");

    return NULL;
}

int main() {

    pthread_t t1;

    pthread_create(&t1, NULL, display, NULL);

    pthread_join(t1, NULL);

    return 0;
}
```

2) pthread_join()

Purpose

Waits for thread completion.

Syntax

- ***pthread_join(thread, NULL);***

3) pthread_exit()

Purpose

Terminates a thread.

Program

```c
#include <stdio.h>
#include <pthread.h>

void* func(void* arg) {

    printf("Thread Started\n");

    pthread_exit(NULL);
}

int main() {

    pthread_t t1;

    pthread_create(&t1, NULL, func, NULL);

    pthread_join(t1, NULL);

    return 0;
}
```

Compilation of pthread Program

Command

The command to compile the pthread program is as follows:

- ***gcc thread.c -o thread -pthread***

Platform

For this assignment the platform that we had used is Kali-Linux.

Software

For this assignment the Software that we had used are as follows:

- *GNU gcc Compiler*
- *POSIX pthread library*
- *Linux Terminal / Command Line*
